

异常、类型与自动化测试

Exception、Type Hint、pytest 和 Unit Test 共同建立快速反馈的质量防线。

- 异常表达运行时失败
- 类型提示表达静态契约
- pytest 组织可执行规范
- 单元测试隔离行为

失败是接口的一部分

可靠接口不仅说明成功返回值，也明确错误类型、上下文和恢复策略。

- 可恢复与不可恢复
- 面向调用方的错误
- 记录诊断上下文
- 避免返回模糊 None

异常层级与边界

在最了解语义的边界转换异常，并通过 `cause` 保留底层诊断信息。

```
try:
    payload = client.fetch()
except TimeoutError as exc:
    raise ServiceUnavailable('upstream timeout') from exc
```

Exception 设计实战

为配置错误、外部服务失败和数据校验失败设计不同的领域异常。

- 名称表达业务语义
- 附带可诊断字段
- 不要捕获 `BaseException`
- `finally` 负责资源清理

Type Hint 与静态反馈

Type Hint 把函数契约前移到编辑和 CI 阶段，但不替代运行时校验。

```
def load_user(user_id: str) -> User | None:
    ...

def require_user(user_id: str) -> User:
    user = load_user(user_id)
    if user is None:
        raise UserNotFound(user_id)
    return user
```

pytest 测试结构

Arrange、Act、Assert 让失败原因清晰，fixture 复用稳定的测试前置。

- 测试名称描述行为
- 每个测试只验证一个原因
- fixture 控制依赖
- 断言输出和副作用

Unit Test 与依赖隔离

单元测试通过替身隔离网络、时间和存储，聚焦当前行为。

```
def test_timeout_is_translated(fake_client):  
    fake_client.fetch.side_effect = TimeoutError()  
    with pytest.raises(ServiceUnavailable):  
        service.load()
```

参数化与失败路径

参数化测试系统覆盖边界输入，并让失败路径与成功路径同等重要。

- 空值
- 边界值
- 非法类型
- 上游超时
- 重复请求

从异常契约到测试契约

每个公开异常都应有对应测试，确保重构不会改变调用方观察到的失败语义。

- 异常类型稳定
- 消息包含关键上下文
- `cause` 被保留
- 日志不泄露敏感信息

质量防线综合练习

为一个外部 API adapter 增加类型、异常转换、pytest 和覆盖失败路径的单元测试。

- 定义公开接口
- 建立领域异常
- 注入 fake client
- 覆盖成功、超时和非法数据